

CONTENTS

Práctico 2 — Análisis de Algoritmos y Orden de Crecimiento	
Programación 3 — FING UdelaR · Resolución completa y verificada	
Ejercicio 1 (Kleinberg & Tardos, Ex. 2.5)	
Ejercicio 2 — Cota $\Omega(n^2)$ para Gale-Shapley	
Ejercicio 3 — Propiedades de o-chica	
Resumen de técnicas usadas en este práctico	

Práctico 2 — Análisis de Algoritmos y Orden de Crecimiento

PROGRAMACIÓN 3 — FING UDELAR · RESOLUCIÓN COMPLETA Y VERIFICADA

Ejercicio 1 (Kleinberg & Tardos, Ex. 2.5)

Dato de partida: $f(n)$ es $O(g(n))$, es decir $\exists c > 0, n_0: \forall n \geq n_0, f(n) \leq c \cdot g(n)$.

(a) $f(n)^2$ es $O(g(n)^2)$ — VERDADERO. Como f, g son positivas, puedo elevar al cuadrado ambos lados de $f(n) \leq c \cdot g(n)$ sin invertir la desigualdad: $f(n)^2 \leq c^2 \cdot g(n)^2$. Tomando $k = c^2$, se cumple $f(n)^2 \leq k \cdot g(n)^2$ para todo $n \geq n_0$ — exactamente la definición de $O(g(n)^2)$. ■

(b) $2^{f(n)}$ es $O(2^{g(n)})$ — FALSO. Contraejemplo: $f(n) = 2n$, $g(n) = n$. Se cumple $f(n) = O(g(n))$ ($2n \leq 2 \cdot n$). Pero $2^{f(n)} = 2^{2n} = 4^n$, mientras $2^{g(n)} = 2^n$. Si 4^n fuera $O(2^n)$, existiría c tal que $4^n \leq c \cdot 2^n \Leftrightarrow 2^n \leq c$ para todo $n \geq n_0$ — imposible, ya que $2^n \rightarrow \infty$. Por lo tanto $2^{f(n)}$ no es $O(2^{g(n)})$. ■

(c) Con la hipótesis extra ($\exists \epsilon > 0: g(n) > 1 + \epsilon$ eventualmente) — VERDADERO. Como $g(n) > 1 + \epsilon > 1$ para n suficientemente grande, $\log(g(n)) > \log(1 + \epsilon) > 0$ — esto es clave, porque me da un piso positivo para dividir. Partiendo de $f(n) \leq c \cdot g(n)$ y aplicando logaritmo (monótono creciente): $\log(f(n)) \leq \log(c \cdot g(n)) = \log(c) + \log(g(n))$

Si $\log(c) \leq 0$ ($c \leq 1$), ya está: $\log(f(n)) \leq \log(g(n))$, tomando $k = 1$. Si $\log(c) > 0$, uso que $\log(g(n)) > \log(1 + \epsilon)$ para acotar el término $\log(c)$ en proporción a $\log(g(n))$: $\log(f(n)) \leq \log(g(n)) + \log(c) \leq \log(g(n)) \cdot (1 + \log(c)/\log(1 + \epsilon))$

Tomando $k = 1 + \log(c)/\log(1 + \epsilon)$ (constante, no depende de n) y n_0 el mismo que en la hipótesis de g , queda $\log(f(n)) \leq k \cdot \log(g(n))$ — es decir, $\log(f(n))$ es $O(\log(g(n)))$. ■

(d) Sin la hipótesis extra ($g(n)$ puede acercarse a 1, o valer exactamente 1) — FALSO. Contraejemplo: $f(n) = 2$ (constante), $g(n) = 1$ (constante). Se cumple $f(n) = O(g(n))$ ($2 \leq 2 \cdot 1$). Pero $\log(f(n)) = \log(2) > 0$ para todo n , mientras $\log(g(n)) = \log(1) = 0$ para todo n . Si $\log(f(n))$ fuera $O(\log(g(n))) = O(0)$, existiría c tal que $\log(2) \leq c \cdot 0 = 0$ — falso, ya que $\log(2) > 0$. Por lo tanto $\log(f(n))$ no es $O(\log(g(n)))$.

(Este es exactamente el caso que la hipótesis de (c) descarta: $g(n) = 1$ no cumple $g(n) > 1 + \epsilon$ para ningún $\epsilon > 0$, por eso (c) necesita esa condición extra y (d) sin ella es falsa.)

Ejercicio 2 — Cota $\Omega(n^2)$ para Gale-Shapley

(a) **Construcción del caso peor.** Tomo $M=\{m_1, \dots, m_n\}$, $W=\{w_1, \dots, w_n\}$ con listas de preferencia:

- m_i prefiere en orden: $w_1, w_2, \dots, w_n$ (todos los hombres tienen la **misma** lista de preferencias).
- w_j prefiere en orden: $m_n, m_{n-1}, \dots, m_1$ (orden inverso — todas las mujeres prefieren a los hombres de mayor índice).

Ejecutando Gale-Shapley (M propone): todos los hombres proponen primero a w_1 . w_1 se queda con m_n (el que más prefiere) y rechaza a los demás $n-1$. Esos $n-1$ hombres proponen a w_2 : w_2 se queda con m_{n-1} (el mejor entre los que le llegaron) y rechaza a $n-2$. Así sucesivamente: en la ronda sobre w_k proponen $n-k+1$ hombres y solo uno se queda.

Cantidad total de propuestas: $n + (n-1) + (n-2) + \dots + 1 = n(n+1)/2$.

Tomando $k=1/2$ y $n_0=0$: $n(n+1)/2 \geq n^2/2$ para todo $n \geq 0$ (ya que $n(n+1) \geq n^2$). Por lo tanto el algoritmo hace $T(n) \geq (1/2)n^2$ propuestas en esta instancia — es decir, $T(n)$ es $\Omega(n^2)$ en el peor caso.

■

(b) **La cota no depende de la implementación.** Sea cual sea la implementación específica de G-S (qué hombre libre se elige en cada paso, cómo se recorren las listas), el algoritmo es **determinista en su comportamiento observable**: para la instancia construida en (a), toda implementación correcta de G-S produce la misma secuencia de compromisos/rechazos (el resultado y la cantidad de propuestas realizadas están determinados por las listas de preferencia, no por detalles de implementación — esto es una consecuencia del teorema de que G-S siempre converge al mismo emparejamiento M-óptimo, sea cual sea el orden en que se procesan los hombres libres).

Por lo tanto, cualquier implementación correcta de G-S sobre esta instancia realiza exactamente $n(n+1)/2 = \Omega(n^2)$ propuestas. Como cada propuesta consume tiempo $\Omega(1)$ (como mínimo, comparar y eventualmente actualizar un compromiso), el tiempo de ejecución de **cualquier** implementación correcta es $\Omega(n^2)$ en esta instancia — independientemente de los detalles de implementación. ■

Ejercicio 3 — Propiedades de o-chica

(a) $T(n)$ es $o(f(n)) \Rightarrow T(n)$ es $O(f(n))$. Por definición de o-chica, la desigualdad $T(n) < \varepsilon \cdot f(n)$ vale para **cualquier** $\varepsilon > 0$ — en particular, para $\varepsilon = 1$: $\exists n_1 \geq 0$ tal que $\forall n \geq n_1$, $T(n) < 1 \cdot f(n) = f(n)$. Tomando $c = 1$ y $n_0 = n_1$, se cumple $T(n) \leq c \cdot f(n)$ para todo $n \geq n_0$ — exactamente la definición de $O(f(n))$. ■

(b) $T(n)$ es $o(f(n)) \Rightarrow \lim_{n \rightarrow \infty} T(n)/f(n) = 0$. Por definición, para cualquier $\varepsilon > 0$ existe n_ε tal que $\forall n \geq n_\varepsilon$: $T(n) < \varepsilon \cdot f(n)$, es decir $T(n)/f(n) < \varepsilon$ (f positiva). Esto es exactamente la definición de límite: para todo $\varepsilon > 0$ existe n_ε tal que para $n \geq n_\varepsilon$ la distancia entre $T(n)/f(n)$ y 0 es menor a ε . Por lo tanto $\lim_{n \rightarrow \infty} T(n)/f(n) = 0$. ■

(c) Ejemplo: $f(n)$ es $O(g(n))$ pero $f(n)$ no es $o(g(n))$. Tomo $f(n) = 2n$, $g(n) = n$. $f(n) = O(g(n))$ con $c = 2$ ($2n \leq 2n$ para todo n). Pero $f(n)$ no es $o(g(n))$: tomando $\varepsilon = 1$, para que valiera se necesitaría $2n < 1 \cdot n$ para todo n suficientemente grande, es decir $n < 0$ — absurdo. Por lo tanto $f(n)$ no es $o(g(n))$. (Caso general: si $f(n) = \Theta(g(n))$ con cociente que no tiende a 0, nunca es $o(g(n))$ — o-chica exige un cociente que se vaya a 0, no que quede acotado.)

(d) $T(n)$ es $o(f(n)) \Rightarrow T(n)$ no es $\Omega(f(n))$. Por absurdo: supongamos $T(n)$ es $o(f(n))$ y $\Omega(f(n))$ a la vez. De Ω : $\exists c > 0, n_0$: $\forall n \geq n_0$, $T(n) \geq c \cdot f(n)$. De o-chica, tomando $\varepsilon = c$ (que es > 0): $\exists n_\varepsilon$: $\forall n \geq n_\varepsilon$, $T(n) < c \cdot f(n)$. Para $n \geq \max(n_0, n_\varepsilon)$ tendríamos simultáneamente $T(n) \geq c \cdot f(n)$ y $T(n) < c \cdot f(n)$ — contradicción. Por lo tanto $T(n)$ no puede ser $\Omega(f(n))$. ■

(e) $T(n)$ es $o(f(n)) \Rightarrow T(n)$ no es $\Theta(f(n))$. $\Theta(f(n))$ exige ser $O(f(n))$ y $\Omega(f(n))$ simultáneamente. Por (d), $o(f(n))$ excluye ser $\Omega(f(n))$. Por lo tanto $o(f(n))$ excluye ser $\Theta(f(n))$ (ya que Θ requiere la parte Ω que acabamos de descartar). ■

(f) Ejemplo: $f(n)$ no es $\Omega(g(n))$ ni $o(g(n))$. Tomo $f(n) = 2$ si n es par, 0 si n es impar; $g(n) = 1$.

- **No es $\Omega(g(n))$:** necesitaría $\exists c > 0, n_0$: $\forall n \geq n_0$, $f(n) \geq c \cdot 1 = c$. Pero para todo n impar, $f(n) = 0 < c$ ($c > 0$) — falla infinitas veces, sin importar n_0 . No es $\Omega(g(n))$.
- **No es $o(g(n))$:** necesitaría que $\forall \varepsilon > 0$, eventualmente $f(n) < \varepsilon$. Pero para todo n par, $f(n) = 2$, así que tomando $\varepsilon = 1$ la desigualdad $f(n) < 1$ falla infinitas veces (en todo n par $\geq$ cualquier n_0). No es $o(g(n))$.

Por lo tanto $f(n)$ no es $\Omega(g(n))$ ni $o(g(n))$ — el comportamiento oscilante de f impide ambas cosas. ■

Resumen de técnicas usadas en este práctico

Ejercicio	Técnica
1	Manipular la definición de O bajo operaciones (cuadrado, exponencial, logaritmo) — cuidado: no todas se preservan igual, y una hipótesis extra puede cambiar $V \rightarrow F$
2	Construcción explícita de instancia de peor caso + argumento de independencia de implementación vía determinismo del comportamiento observable
3	Manipulación directa de cuantificadores ($\forall \epsilon \exists n_\epsilon$) para pasar de o a O , a límite, y a exclusión mutua con Ω/Θ

Para el examen: este práctico es sobre todo un ejercicio de manejo formal de cuantificadores — vale la pena poder escribir de memoria, sin dudar, las 4 definiciones (O , Ω , Θ , o) con sus cuantificadores en el orden correcto, porque son la base de **todas** las demostraciones de complejidad del resto del curso.